

Business Analyst Intern Technical Assessment

Question Set: A — Answers Document

Submitted by: Happy Kumar

Sections 1–3 covered in this document. Section 4 (Mini-Project) is delivered separately in the project/ folder.

Section 1 — Data Quality Checks

Footnote 7 — Loyalty Tier 'None' Parsing Trap:

The CSV was loaded with `keep_default_na=False` and `na_values=[]`, which is required to correctly preserve the literal string 'None' in `customer_loyalty_tier` as a valid tier value rather than a missing value. Verified: with the default pandas reader, 5,571 rows (46.4%) of `customer_loyalty_tier` silently become NaN; with the corrected reader, those same rows correctly show as the string 'None'. This fix was applied before all checks below.

A1. Invalid Stays

Question: How many bookings have invalid stays (checkout date $\leq$ check-in date)?

Answer: 120 invalid stays

Computed by filtering rows where `checkout_date <= checkin_date` (Footnote 1). This represents 1.0% of the 12,000-row dataset — consistent with an injected data-error rate, not a systemic issue.

A2. Review Rating by Segment

Question: Compute review rating range and mean separately for each customer segment. What does this reveal?

Footnote 6 requires normalization before any cross-segment comparison: Corporate reviews are recorded on a 1–10 scale, while Individual and Group use 1–5. Corporate ratings were divided by 2 to bring all segments onto a common 1–5 scale.

Segment	Min	Max	Mean (normalized)	Count
Corporate	1.5	5.0	3.62	1,287
Group	1.0	5.0	3.77	590
Individual	1.0	5.0	3.77	3,269

1-line implication:

After normalizing Corporate ratings to the 1–5 scale, mean satisfaction is nearly identical across all three segments (~3.6–3.8); without normalization, Corporate's raw mean would appear roughly double the others purely due to scale, falsely implying a segment-driven rating gap that does not actually exist.

A3. Realized Revenue — Luxury Properties

Question: After applying the relevant data-quality footnotes, what is realized revenue for property type = Luxury?

Three footnotes applied before aggregating: excluded invalid stays (Footnote 1: `checkout ≤ checkin`), excluded zero-room bookings (Footnote 3: `num_rooms = 0`, which always carry `total_amount = 0` and are erroneous), and kept only `booking_status = 'Completed'` (Footnote 8: Cancelled/No-Show rows carry a would-have-been-charged amount, not realized revenue).

Answer: ₹90,694,052.93

Validation: summing `total_amount` across all Luxury rows regardless of status (no filters applied) gives ₹117,431,484.47 — a 29.5% overstatement, closely matching Footnote 8's predicted ~30% overstatement. This confirms the filtering logic correctly isolates realized revenue.

Additional Footnote Verification (FN2, FN5)

Two footnotes are not directly tied to a specific Section 1 question but were checked for completeness, since the brief warns these are real data quirks that will sink surface-level analysis.

Footnote 5 — Reviews on Cancelled bookings:

50 rows have `booking_status = 'Cancelled'` but carry a non-null `review_rating`, which is logically impossible. These rows were not excluded from the A2 segment-level rating calculation, since Footnote 6 (scale normalization) is the only adjustment that question explicitly calls for. Flagged here for visibility — a stricter analysis should exclude these 50 rows before computing review averages.

Footnote 2 — Booking before signup:

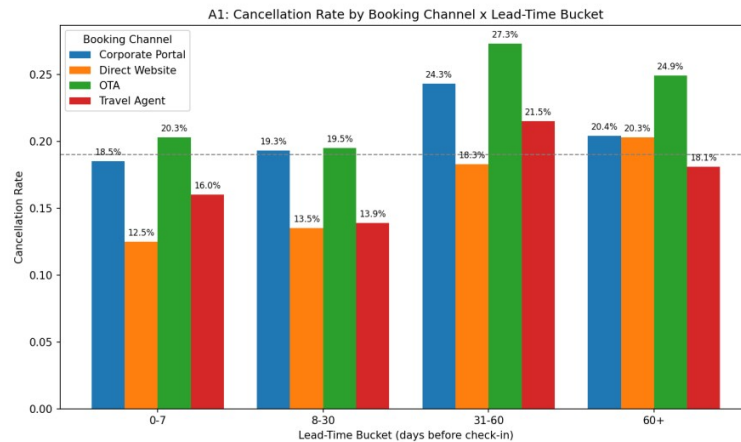
163 rows have `booking_date` earlier than `customer_signup_date` — a temporal-integrity issue. Checked for overlap with other flagged cohorts: only 3 of these 163 are also invalid stays (FN1), and 125 are Completed bookings, meaning they are not automatically excluded from A3's realized revenue figure. This is a known limitation of the ₹90,694,052.93 figure — a small number of bookings with this temporal anomaly may be included.

Section 2 — Case Study: The Cancellation Crisis

Briefing: platform-wide cancellation rate is ~19% (verified: 19.15% on the cleaned dataset of 11,821 bookings, after excluding invalid stays and zero-room rows). Board target: reduce by 5 percentage points to ~14% within two quarters.

A1. Cancellation Landscape

Cancellation rate was broken down by booking_channel x lead_time_bucket (lead time = days between booking_date and checkin_date, bucketed into 0-7, 8-30, 31-60, 60+ days).



Channel	0-7	8-30	31-60	60+
Corporate Portal	18.5%	19.3%	24.3%	20.4%
Direct Website	12.5%	13.5%	18.3%	20.3%
OTA	20.3%	19.5%	27.3%	24.9%
Travel Agent	16.0%	13.9%	21.5%	18.1%

Which dimension drives the most variance:

Booking channel is the dominant driver. OTA shows the highest or near-highest cancellation rate in every lead-time bucket, while Direct Website shows the lowest in every bucket — a consistent ordering across all four buckets, not a one-off spike. Lead-time bucket is a secondary, amplifying factor: cancellation rate rises in the 31-60 day window across nearly all channels, peaking there before easing slightly at 60+. The single worst slice is OTA x 31-60 days at 27.3%; the best is Direct Website x 0-7 days at 12.5%.

A2. Rate vs. Volume

Rank	Top 3 by RATE	Bookings	Cancellations	Rate
1	OTA x 31-60	943	257	27.3%
2	OTA x 60+	370	92	24.9%
3	Corporate Portal x 31-60	675	164	24.3%
Rank	Top 3 by COUNT	Bookings	Cancellations	Rate
1	OTA x 8-30	1,496	291	19.5%
2	OTA x 0-7	1,375	279	20.3%

Rank	Top 3 by RATE	Bookings	Cancellations	Rate
3	OTA x 31-60	943	257	27.3%

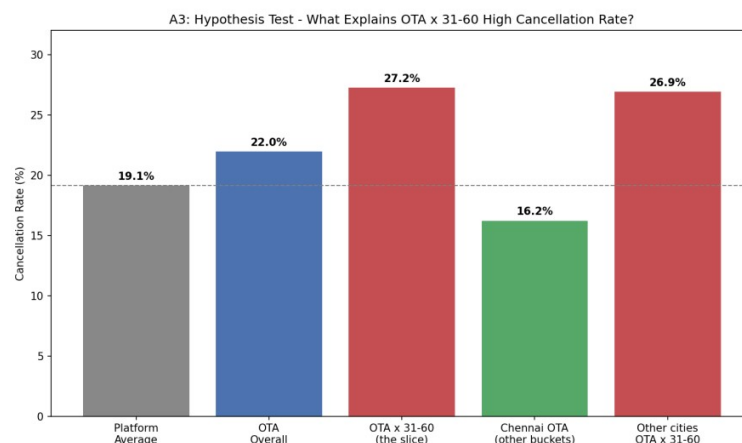
Why this matters for the board's 5pp target:

Only OTA x 31-60 appears on both lists. The other rate-leaders (OTA x 60+, Corporate Portal x 31-60) are low-volume (370 and 675 bookings) — fixing them barely moves the platform number. The volume-leaders (OTA x 8-30, OTA x 0-7) have only moderate rates (~19-20%, near the platform average) but their sheer size (1,375-1,496 bookings each) makes them the larger source of absolute cancellations. A 5pp platform-wide reduction requires moving total cancellation count, not just flattening a few extreme percentages — targeting OTA broadly has far more leverage than targeting only the worst-rate slices.

A3. Root Cause: Hypothesis Testing

Worst-rate, high-volume slice selected for testing: OTA x 31-60 days (943 bookings, 257 cancellations, 27.25% rate).

Hypothesis	Comparison	Result
H1: Lead-time effect	Slice mean lead-time (48.9 days) vs rest of platform (31.1 days)	Inconclusive — circular by definition, since the slice IS the 31-60 day bucket
H2: Channel-mix effect	OTA overall rate (21.96%) vs OTA within this bucket (27.25%)	Real signal — 5.3pp jump within the same channel, isolated to this lead-time window
H3: City-season effect	Chennai OTA, other buckets (16.2%) vs Other cities, OTA x 31-60 (26.9%) vs the slice itself (27.25%)	Ruled out — effect persists across cities; Chennai alone drops to near-average outside this window



Conclusion:

The data best supports an OTA x 31-60-day lead-time interaction effect — not a pure channel effect, not a pure lead-time effect, and not a city/season effect. OTA bookings made 31-60 days before check-in are the specific risk pocket, holding consistently across markets (Chennai and non-Chennai cities show nearly identical elevated rates within this window).

A4. Targeted Recommendation

WHO:

OTA bookings with 31-60 day lead time — 943 bookings, 257 cancellations, 27.25% cancel rate (the single worst-rate slice that is also high-volume, per A2/A3).

WHAT:

A 15-20% non-refundable deposit collected at booking time, applied only to this slice (OTA + 31-60 day lead time). A deposit is preferred over an outright non-refundable policy because OTA is a volume-driving channel — an overly aggressive policy risks suppressing bookings, not just cancellations.

QUANTIFIED IMPACT:

Baseline: 943 bookings, 257 cancellations, 27.25% rate. Target range: reduce to between platform average (19.15%) and OTA's own overall baseline (21.96%).

- Lower bound: $943 \times (27.25\% - 21.96\%) = 943 \times 0.0529 \approx 50$ fewer cancellations
- Upper bound: $943 \times (27.25\% - 19.15\%) = 943 \times 0.0810 \approx 76$ fewer cancellations

Platform-wide denominator: 11,821 clean bookings, current total cancellations = 2,264 (19.15%).

- Lower bound impact: $50 / 11,821 \approx 0.42$ percentage points
- Upper bound impact: $76 / 11,821 \approx 0.64$ percentage points

Expected platform-wide impact: approximately 0.4-0.6 percentage points

(cancellation rate moves from 19.15% to roughly 18.5-18.7%). This single intervention is insufficient alone to close the board's full 5pp gap, since this slice is only 943 of 11,821 bookings (~8.0% of volume). Reaching the full target would require applying similar targeted interventions to the next-worst slices (e.g., Corporate Portal x 31-60, at 24.3%) in addition to this one.

RISK / collateral effect:

A deposit requirement may suppress OTA conversion/bookings — the same price-sensitive, comparison-shopping customer behavior that drives high cancellation may also make this segment deposit-averse, trading a cancellation problem for an acquisition problem.

Section 3 — SQL Challenge

Full schema CREATE TABLE statements and complete SQL queries are provided as plain text in the separate file `sql_section3.docx` (included in this submission). This section gives a brief summary of the design and results.

Dialect note: queries were designed conceptually against the normalized schema described below, but executed against the flat CSV loaded as a single table (`bookings_flat`) in an in-memory SQLite database via Python (`sqlite3` + `pandas`), as permitted by the assessment instructions.

Schema Design — Summary

Four normalized tables were designed: `customers`, `properties`, `bookings`, `reviews`. Each column specifies a SQL data type, primary/foreign key status, NOT NULL constraints where appropriate, and at least one CHECK/UNIQUE constraint tied to a footnote (see `sql_section3.docx` for full CREATE TABLE statements).

- `total_amount` and `adr` use DECIMAL, not FLOAT — money should never use floating point due to rounding errors.
- `bookings.CHECK` (`checkout_date > checkin_date`) directly enforces Footnote 1 at the schema level — invalid stays become structurally impossible going forward, even though the raw CSV has 120 of them.
- `properties.UNIQUE`(`property_name`, `property_city`, `property_id`) addresses Footnote 4 — property names repeat across cities with different IDs (e.g. 'The Grand Plaza' exists in both Bangalore and Kochi as two distinct properties), so `property_id` is the true uniqueness anchor, never `property_name` alone.
- `reviews` is a separate table since a review is optional and 1:1 with a booking — UNIQUE on `booking_id` enforces that. Application-level logic should prevent inserting a review for a Cancelled booking, addressing Footnote 5.
- `customer_loyalty_tier` defaults to and explicitly allows the literal string 'None' (Footnote 7) rather than a NULL, preserving the distinction between 'no tier' (valid value) and missing data.

SQL Queries — Summary

A-Q1: Highest Realized Revenue per City

`RANK()` OVER (`PARTITION BY property_city ORDER BY revenue DESC`), revenue from Completed bookings only (Footnote 8), grouped by `property_id` not `property_name` (Footnote 4). Full query text in `sql_section3.docx`.

Result: 10 rows (one per city).

Notable: 'The Grand Plaza' is the top property in both Bangalore (`property_id` 2) and Kochi (`property_id` 1) — two distinct properties sharing a name, confirming Footnote 4 and validating that grouping by `property_id` was essential.

A-Q2: Customers with Average Booking Gap Under 30 Days

`LAG()` OVER (`PARTITION BY customer_id ORDER BY checkin_date`), restricted to Completed bookings, computes the gap in days between consecutive bookings per customer, averaged per customer. Full query text in `sql_section3.docx`.

Answer: 445 customers

have an average gap under 30 days between consecutive Completed bookings.

Dialect note: SQLite has no native date-subtraction operator, so `julianday()` converts dates to numeric day values before subtracting.

Section 4 — Mini-Project Summary

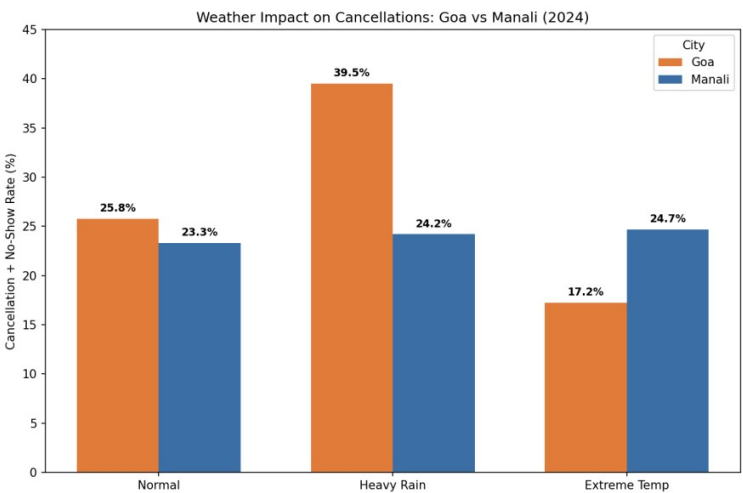
Full working code, README, and AI-usage note are delivered separately in the project/ folder, as required. This page summarizes the build and the core insight for reference.

What Was Built

A Weather-Cancellation Analyzer for Set A: fetched real 2024 daily weather (precipitation, max temperature) for Goa and Manali from the Open-Meteo Historical Weather API (archive-api.open-meteo.com/v1/era5), merged it onto each booking's check-in date, and tested whether heavy rain or extreme temperature correlates with cancellation/no-show rates.

- Scope: 1,604 bookings (891 Goa, 713 Manali), all 2024 check-ins
- API calls: 2 total (one per city, full-year date range) — batched per the assignment's 'do not hit the API once per row' requirement
- Merge: joined on (property_city, checkin_date) after date normalization — 0 unmatched rows out of 1,604
- Error handling: try/except covering Timeout, RequestException, and malformed JSON response, so a network failure would not crash the run

The Insight



Heavy rainfall on the check-in date is associated with a substantially elevated cancellation and no-show rate in Goa specifically — 39.5% on Heavy Rain days versus 25.8% on Normal days, a 13.7 percentage point increase — but shows no meaningful effect in Manali (24.2% vs 23.3%, a negligible 0.9pp difference). This city-specific pattern would have been invisible without merging live weather data: the dataset alone has no way to distinguish a coincidental cancellation cluster from a genuine weather-driven pattern, let alone reveal that the effect is concentrated in one destination and absent in the other. The likely explanation is that Goa's core appeal (beach, water sports, outdoor dining) is directly disrupted by rain, while Manali's appeal (mountains, snow, scenery) is comparatively weather-resilient — suggesting Goa-specific rain-contingent cancellation policies would be a more targeted intervention than a platform-wide weather policy.